Promo Finder - how it works

A plain-language guide for someone who knows programming but has not worked much with AI APIs.

1. The problem

A microdrama is a show with very short episodes (1-2 minutes). To get new viewers, the company posts **promos**: 20-60 second clips cut from the show, on Instagram, TikTok and YouTube. Today an editor watches every episode and picks these clips by hand. This tool reads the **screenplays** (the written scripts, where every line has a timestamp) and finds the clips automatically.

A good promo has three things:

- **A hook** in the first 3 seconds - something that makes a scrolling person stop (a slap, a shocking line).
- **A cliffhanger** at the end - it stops at an open moment, so the viewer wants to know what happens next.
- **It makes sense alone** - a stranger who never saw the show can follow it.

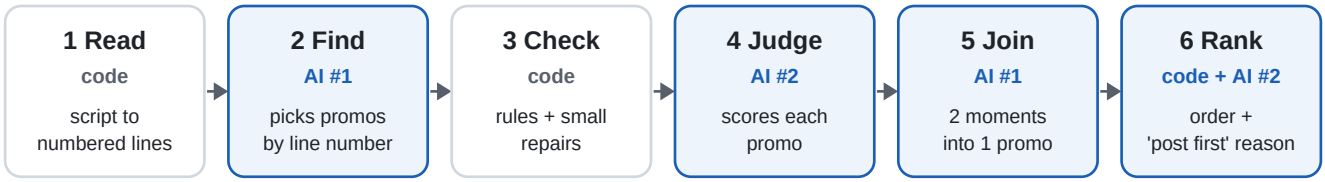
2. What the tool gives you

A ranked list. For each promo: where it is (episode, start and end time), the hook, the cliffhanger, why it works, and why to post it at this position. For example (made-up lines, not from the real show):

```
#1  Episode 2  00:18.20 - 00:52.40  (34 s)  quality 4/4
    Opens on:  "You promised to marry me in front of everyone."
    Ends on:   "Then who is the woman wearing my ring?"
    Why it works:  instant betrayal, and the last line is an unanswered question.
    Why post it first:  sells the premise of the whole show in one clip.
```

3. The big picture

The work is split into six steps. Two different AI models do the parts that need judgement; normal code does everything that can be checked by rules.



Blue = an AI model does this step. White = plain Python code does it (fast, free, always the same).

> **The most important design idea**
>
> The AI never writes times or dialogue. Every script line gets a number (L12, L13 ...). The AI answers only with line numbers ("start at L12, hook is L12, cliffhanger is L30"), and the code looks up the real text and the real times. So the AI cannot invent a line or a wrong timestamp - the worst it can do is pick a bad line, and the code then checks that.

4. AI APIs in five minutes

An **LLM** (large language model) is a program that reads text and writes text. You do not run it yourself; a company runs it on their servers, and you use it through an **API**: your code sends an HTTP request over the internet, and gets an answer back - just like calling any web service.

What a request looks like

Almost every provider accepts the same format, called the **OpenAI-compatible format**. You send a JSON body with the model name and the messages:

```
POST https://api.groq.com/openai/v1/chat/completions
Authorization: Bearer <your API key>

{ "model": "openai/gpt-oss-120b",
  "messages": [
    {"role": "system", "content": "You pick promo clips. Reply ONLY with JSON."},
    {"role": "user",   "content": "L1 [0.0s-2.0s] JULIA: \"You knew?\" ..."} ],
  "response_format": {"type": "json_object"} }
```

The answer comes back as JSON too; the model's text is inside it:

```
{ "choices": [ {"message": {"content": "{\"promos\": [{\"hook_line\": \"L1\", ...}]}"},
                "finish_reason": "stop"} ],
  "usage": {"prompt_tokens": 1830, "completion_tokens": 410} }
```

The words you need

Word	What it means
API key	A secret password for your account. Sent with every request. Never put it in the code - this project keeps keys in a **.env** file that is not shared.
Provider	The company running the model: Groq, OpenAI, Anthropic, Google (Gemini), Amazon Bedrock, OpenRouter ...
Model	The specific AI, e.g. *gpt-oss-120b* or *deepseek-v3.2*. Bigger models are usually smarter, slower and cost more.
System / user message	The **system** message says what the model's job is and the rules; the **user** message is the actual input (here: one episode's script).
Prompt	All the text you send. This project keeps every prompt in one file: **promo/prompts.py**.
Token	Models count text in tokens (about 3/4 of a word). You pay per token: input tokens + output tokens.
JSON mode	Asks the model to reply with valid JSON only, so code can read the answer. Not every model supports it.
Rate limit / quota	Free plans limit how much you can send: per minute (wait and retry) and per day (stop, continue tomorrow).
Reasoning model	A model that "thinks" first in hidden text before answering (e.g. gpt-oss, o3, DeepSeek-R1). Better at hard tasks, but the thinking also uses tokens.

> **Why answers must be JSON**
>
> The code has to read the model's answer automatically. Free text like "I think the best clip starts at line 12" is hard to use; JSON like {"start_line": "L12"} is easy. So every prompt in this project asks for JSON and shows the exact shape it wants.

5. Talking to any model: promo/llm.py

All communication with AI models goes through one small file. The rest of the project only calls one function: **ask_json(system, user)**, which returns a Python dictionary. That is why the model is easy to swap.

Choosing a model = one line

A model is written as **provider/model-name**. The file already knows the web address, the key name and the request style of the common providers, so you only change this line in **config.toml**:

```
model  = "groq/openai/gpt-oss-120b"     # the model that finds promos
critic = "groq/openai/gpt-oss-20b"      # a DIFFERENT model that checks them
```

and put the key in **.env** (for example GROQ_API_KEY=...). Built in: openai, anthropic, groq, gemini, openrouter, bedrock, deepseek, mistral, together, ollama (runs locally, no key), and **custom** for any other OpenAI-style API. Before a full run you can test a model in about 10 seconds:

```
python3 -m promo.llm check groq/openai/gpt-oss-120b
```

What goes wrong with real models - and what the file does

When we switched between models, each one broke in a different way. These problems all really happened in this project; the file now handles each of them, so a new model usually just works:

What went wrong	Seen with	What llm.py does
The answer stopped half-way: the reasoning model used all its tokens on hidden thinking	Groq gpt-oss-20b, Bedrock gpt-oss-120b	Sets a reasoning level; if an answer is cut off, doubles the token limit and asks again
Thinking text mixed into the answer (<reasoning> ... </reasoning>), even with JSON-looking text inside it	Bedrock gpt-oss	Removes the thinking part before reading the JSON
Messy JSON: in code fences, with chatty text, a doubled opening brace, trailing commas	Bedrock, Nemotron	A tolerant reader that finds the first valid JSON object
The server refuses an option (JSON mode, reasoning setting) with error 400	Groq, OpenRouter	Drops that option and sends the request again
Valid JSON, but not the shape we asked for	smaller models	One 'repair' message: "reply with ONLY the JSON object"
Rate limit (error 429): "try again in 12 s"	every free plan	Waits as long as the server says, then retries
Wrong key or model name (401/403/404)	-	Stops at once with a clear message (retrying would not help)

Two more things that save time and money

- **Cache.** Every answer is saved in **outputs/<run>/raw_llm/**. Running the same command again re-uses the saved answers and only asks for what is missing - so if the quota runs out half-way, you just run it again later. A finished show re-builds in under a second, with no key.
- **Parallel calls.** Six episodes are processed at the same time (threads), with a cap of 16 calls at once so the provider is not flooded.

6. The steps, with one example

Step 1 - Read (code). promo/parse.py turns each script file into a list of lines with a number, a type (dialogue, action, sound, camera) and start/end times. What the model sees:

```
L12 [33.0s-35.1s] MARGARET (Cold): "You should turn around."
L13 [35.5s-36.3s] JULIA (Confused): "Why?"
L14 [36.4s-38.0s] ACTION: Julia sees the groom holding another woman's hand.
L15 [38.2s-39.0s] MARGARET: "Not anymore."
```

Step 2 - Find (AI model 1). For each episode the model gets the whole script plus a short **show bible** (who is who, and which twists come late - so it can avoid spoilers). It answers with up to 6 promos, using line numbers only:

```
{"promos": [{"segments": [{"start_line": "L12", "end_line": "L30"}],
             "hook_line": "L12", "cliff_line": "L30", "why": "public betrayal ..."}]}
```

Step 3 - Check (code). promo/validate.py looks up the lines and checks the brief's rules. Small mistakes are repaired (and the repair is written down); the rest are thrown away with a reason:

- Do the line numbers exist? (a model once answered "L40.5")
- Is it 20-60 seconds? A clip 2 s too long can lose one middle line; a clip too short can get a short setup line in front.
- Is the hook in the first 3 seconds? If not, the start is moved.
- Does it END on the cliffhanger? Lines after it are cut off.
- Is the cliffhanger a real story moment? A sound effect or a look ("she gasps") is not - the clip then ends on the last spoken line.
- Is the first line vague, like "Look at him."? Then it is flagged as a weak opening.

Step 4 - Judge (AI model 2). A **different** model checks every clip - the model that picked the clips should not grade its own work. It answers yes/no questions (does the opening show a conflict? is the last line an open question? would a stranger follow it? does it spoil a late twist?) and votes 2-3 times, because one answer can be noisy. A second call, the **cold viewer**, gets only the clip - no background at all - and says who the conflict is between and whether it would keep watching. The result is a score from **0 to 4**: hook, ending, understandable alone, not a spoiler.

Step 5 - Join (AI model 1). Sometimes the best hook and the best cliffhanger are too far apart for one clip. The code collects strong moments and asks the model to join 2-3 of them, from one episode (type B) or from different episodes (type C). These go through Steps 3 and 4 like every other promo.

Step 6 - Rank (code + AI model 2). promo/rank.py sorts by the 0-4 score first, so a weaker clip is never placed above a stronger one. Inside the same score: clean openings before weak ones, non-spoilers first, earlier episodes first. It removes duplicates (the same moment twice), allows at most 3 clips of the same story type in the top 10, and asks the model to order small groups and say *why to post it first*.

7. Running it

```
# once: install
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
# find the promos (models from config.toml)
python3 run.py --input "../I Married a Broken Soldier. He was a Duke"
# open the editor screen in the browser
.venv/bin/streamlit run app.py
# compare two versions
python3 eval/score.py outputs/<old run> outputs/<new run>
```

The results are in **outputs/<model>/**: **promos.md** (read this first), promos.csv, promos.json, **rejected.json** (what was thrown away and why), run_stats.json (counts, time, cost) and raw_llm/ (every model answer, unedited). The **app** is the editor's screen, with 4 tabs: **Best promos** (one card per promo; the editor clicks **Post / Maybe / Skip**), **Where it struggles**, **Read an episode** and **How it works**. The editor's choices are saved and used to measure quality - this is where a human stays in the loop.

8. Does it work? Real numbers from the full show (50 episodes)

Problem in the top promos	First version	Final version
Clip keeps running after the cliffhanger (top 40)	4	0
Clip ends on a sound effect (top 40)	2	0
The same moment twice (top 40)	1	0
Vague opening line (top 10)	2	0
Hook or ending is only a look / gasp (top 10)	1	0

Final run: 165 proposals, 156 passed the code checks, 60 duplicates removed, 40 ranked, covering 33 episodes. The top 10 average 3.7 out of 4 on the critic's score. Finding the promos cost about **$0.07** for the whole show (gpt-oss-120b on Amazon Bedrock); the critic used about 1.7 million tokens; a full run takes about 10-20 minutes with 6 episodes in parallel, and seconds when cached. The same setup also works on free plans (Groq), just slower because of daily limits.

9. Where it is weak

- It only reads the script. A hook that is purely visual (a look, a costume) can be scored wrong.
- The critic is also an AI, so it can be wrong. That is why the editor's Post / Maybe / Skip is the real test.
- Promos joined from different episodes usually repeat a moment that a normal promo already uses, so most are removed as duplicates.
- The list of "real action" words (strikes, grabs, enters ...) that decides if an action line can end a promo is written by hand.

10. Where to look in the code

File	Job	File	Job
run.py	runs all six steps, top to bottom	promo/critic.py	step 4: the second model's score
promo/parse.py	step 1: reads the scripts	promo/stitch.py	step 5: joins moments
promo/prompts.py	every instruction sent to the AI	promo/rank.py	step 6: order, duplicates, variety
promo/llm.py	talks to any model (page 3)	app.py	the editor's screen
promo/validate.py	step 3: the rules and repairs	eval/score.py	measures quality

Tip: to see the whole flow, read **run.py** from top to bottom - its five numbered sections match the steps in this guide.